

Topic 4: **Testing the Code**



**UNIVERSITY OF
MPUMALANGA**



Introduction

- Quality of the code is one of the main requirements for releasing the code
 - Testing is the requirement for quality assurance
- Two of the vital testing framework for a software are:
 - JUnit for unit testing and
 - Selenium for web frontend testing.



Manual Testing

- Automation is the main goal of DevOps,
 - However, manual testing will always be vital in software quality attainment
- The requirement is to have a minimum of one manual test before any automated test
- Both manual and automated testing have the same dependent variables as follows:
 - Manage test data so that tests give the same results when you run them repeatedly
 - Be able to make rapid deployments of new code in order to verify bug fixes
- There are cases when the personnel details have to be removed before deploying to test environment.



Prons and Cons

- Compilation alone is not enough, testing is necessary to ensure reliability.
- The concept of automation is sort after because it has a potential to assure
 - Higher software quality,
 - Higher confidence, and
 - Less time waist
- However, experience shows that most organizations talk about automated testing while few companies have test automation in practice.

Prons and Cons

Test Automation Problems



- Automated Unit Testing is a cheap and have lower values
 - Manual testing is more valuable (exposes more bugs)
- Test cradles for automated integration test are hard to create
 - Its easier for unit test but becomes difficult as test cradle are production like
- Tests must be adjusted (hard) proportional to functional change over time
- Robust test are hard to write that work well in different build scenarios.
- It is just hard to write good automated tests, period.

Unit Testing

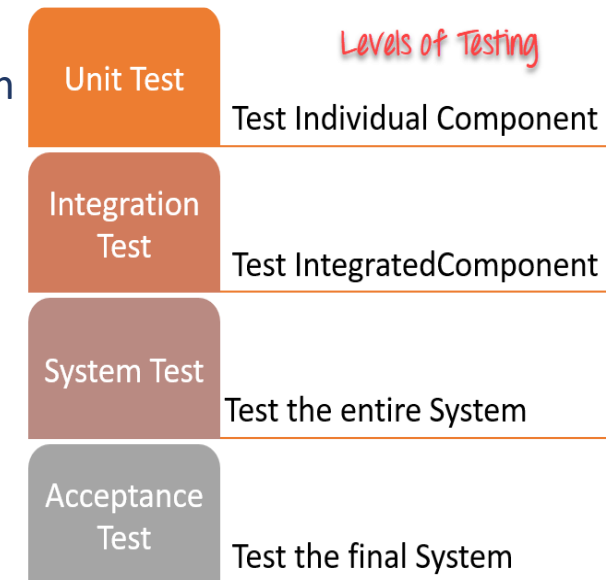


What is Unit Test?

- It is a piece of code written by a developer to test an individual unit of a code

What is the goal of Unit test?

- It is to validate that each unit of the software performs as designed.
- A unit, is the smallest testable part of any software.
 - In procedural programming: a unit is an individual program, function, procedure, etc.
 - In object-oriented programs: the smallest unit is a method
- Developers prefers Unit testing because it tests well-defined parts of the system in isolation from other parts.
 - Easy to write and use.
- Sometimes Unit testing can be confused with functional test
 - However, unit testing is not concerned with formal validation and verification or correctness or performance testing.
 - Unit test is client negligent

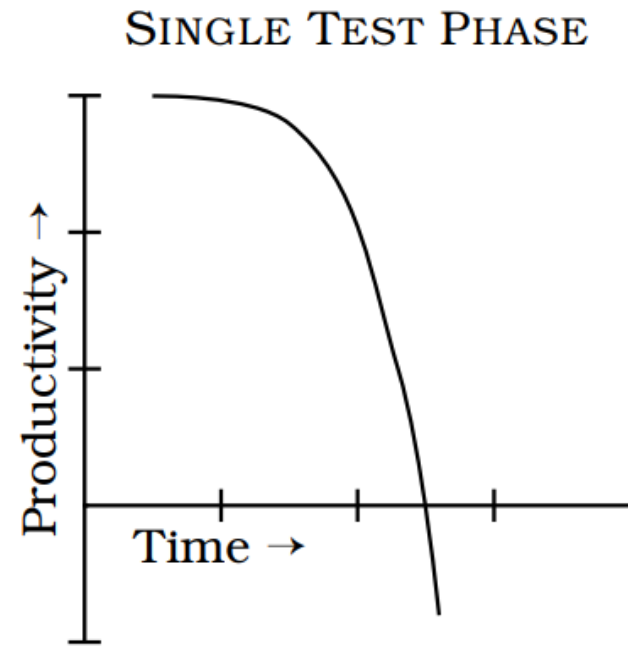
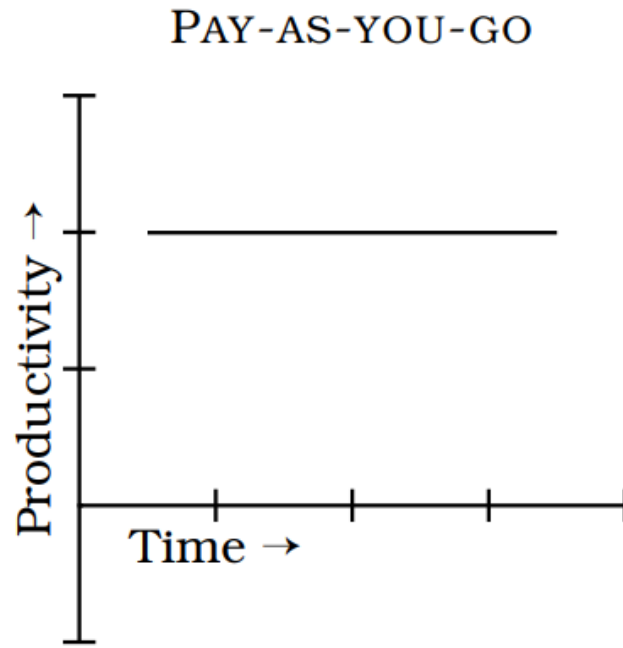


Why a Unit Test?



- Unit testing will make your life easier.
- It drastically reduce the amount of time you spend debugging
- It will make your designs better

Test Phases Comparison



It takes too much time to write the tests

- No1 excuse

Its true, it takes too much time, if?.....



- Many people view testing of any sort as something that happens at the end
- Bea approach is to write individual tests with the code itself as you go along

Tutorial: **first Unit Test**



**UNIVERSITY OF
MPUMALANGA**



Introduction

- To check if code is behaving as you expect, you use an assertion
 - An assertion is a simple method call that verifies that something is true.
- Example: the method **assertTrue** checks that the given boolean condition is true, and fails the current test if it is not.

```
public void assertTrue(boolean condition) {  
    if (!condition) {  
        abort();  
    }  
}
```

- You could use this assert method to check many things, including whether numbers are equal to each other:

```
int a = 2;  
assertTrue(a == 2);
```

```
public void assertEquals(int a, int b) {  
    assertTrue(a == b);  
}
```



Planning Tests

- Lets start with a single static method that find the largest number in a list
 - `int Largest.largest(int[] list);`
 - Assume the list has the following: [7, 8, 9] the test should return 9
- What are other combinations of list that one can have?
 - BasicTest: test if it returns the expected output under normal input
 - StringTest:
 - lengthTest:
 - orderTest:
 - negativeTest:



Planning Tests

- Lets start with a single static method that find the largest number in a list
 - `int Largest.largest(int[] list);`
 - Assume the list has the following: [7, 8, 9] the test should return 9
- What are other combinations of list that one can have?
 - Order List Test
 - [7, 8, 9] → 9
 - [8, 9, 7] → 9
 - [9, 7, 8] → 9
 - Duplicate List Test
 - [7, 9, 8, 9] → 9
 - Single entry Test
 - [1] → [1]
 - Negative number list Test
 - [-9, -8, -7] → -7



Exercise

- Write a method that returns a largest number

<https://www.youtube.com/watch?v=2nWI8wZM-U0>

<https://github.com/junit-team/junit4/wiki/Download-and-Install>



Exercise

```
public class Largest {  
    public static int largest(int[] list) {  
        int index, max=Integer.MAX_VALUE;  
        for (index = 0; index < list.length-1;  
            index++) {  
            if (list[index] > max){  
                max = list[index];  
            }  
        }  
        Return max;  
    }  
}
```



Testing the Method:Test1

```
import junit.framework.*;
public class TestLargest extends TestCase{
    public TestLargest(String name){
        super(name);
    }
    public void testLargestSimple() {
        assertEquals(9, Largest.largest(new int[] {7,8,9}));
    }
}
```

Java -cp junit-4.13.jar;hamcrest-core-1.3.jar;. Org.junit.runner.JUnitCore testfile



Testing the Method: Test 1 Alternative

```
import junit.framework.*;
public class TestLargest extends TestCase{
    public TestLargest(String name){
        super(name);
    }
    public void testLargestSimple() {
        int[] arr = new int[3];
        arr[0] = 7;
        arr[1] = 8;
        arr[2] = 9;
        assertEquals(9, Largest.largest(arr));
    }
}
```



Testing the Method:Test2

```
import junit.framework.*;
public class TestLargest extends TestCase{
    public TestLargest(String name){
        super(name);
    }
    public void testLargestOrder(){
        assertEquals(9, Largest.largest(new int[] {9,8,7}));
        assertEquals(9, Largest.largest(new int[] {7,9,8}));
        assertEquals(9, Largest.largest(new int[] {7,8,9}));
    }
}
```



Testing the Method:Test3

```
import junit.framework.*;
public class TestLargest extends TestCase{
    public TestLargest(String name){
        super(name);
    }
    public void testLargestDups() {
        assertEquals(9, Largest.largest(new int[] {9,7,9,8}));
    }
}
```



Testing the Method:Test4

```
import junit.framework.*;
public class TestLargest extends TestCase{
    public TestLargest(String name){
        super(name);
    }
    public void testLargetOne(){
        assertEquals(1, Largest.largest(new int[] f1g));
    }
}
```



Testing the Method:Test5

```
import junit.framework.*;
public class TestLargest extends TestCase{
    public TestLargest(String name){
        super(name);
    }
    public void testLargestNegative() {
        int [] negList = new int[] {-9, -8, -7};
        assertEquals(-7, Largest.largest(negList));
    }
}
```

Tutorial: **first JUnit Test**



**UNIVERSITY OF
MPUMALANGA**



Naming Rules (Conventions)

TestAccount.java

```
testCreateAccount()  
testCreateAcctDef()  
testCreateAcctDup()
```

(Internal Only)

Account.java

```
createAccount()
```

(Delivered)

Test code have four functions:

- Setup all conditions needed for testing
 - **create any required objects, allocate any needed resources, etc.**
- Call the method to be tested
- Verify that the method to be tested functioned as expected
- Clean up after itself



JUnit Asserts

- Run the test code,
 - The test code will indirectly run or exercise the production code

<https://github.com/junit-team/junit4/wiki/Download-and-Install>



JUnit Asserts

- A sample of JUnit assertion methods available:
 - To check whether two objects are equal:
 - `assertEquals(str1, str2);`
 - To check whether a condition is true:
 - `assertTrue (val1 < val2);`
 - To check whether a condition is false:
 - `assertFalse(val1 > val2);`
- The whole list of JUnit assertion methods can be found in this link:
 - <https://junit.org/junit4/javadoc/latest/org/junit/Assert.html>
- All assert methods will have three options outputs and report them through the JUnit classes. The output options are:
 - Failures: results from a false assertion & current (not all) test method is aborted
 - Errors: results from an unexpected exception & current (not all) test method is aborted
 - Success: results from a true assertion & execution of the current test method is aborted



assertEquals

`assertEquals ([String message], expected, actual)`

- This is the most-often used form of assert.
 - *expected* is a value you hope to see (typically hard-coded),
 - *actual* is a value actually produced by the code under test.
 - *message* is an optional message that will be reported in the event of a failure.
- You can omit the message argument

`assertEquals (expected, actual)`

assertNull



`assertNull([String message], java.lang.Object object)`

`assertNotNull([String message], java.lang.Object object)`

- Asserts that the given object is null (or not null), failing otherwise. The message is optional.



assertSame

`assertSame([String message], expected, actual)`

- Asserts that expected and actual refer to the same object, and fails the test if they do not. The message is optional

`assertNotSame([String message], expected, actual)`

- Asserts that expected and actual do not refer to the same object, and fails the test if they are the same object. The message is optional.



assertTrue

`assertTrue([String message], boolean condition)`

- Asserts that the given boolean condition is true, otherwise the test fails. The message is optional.



JUnit Framework

```
Line 1  import junit.framework.*;
Line 2  public class TestSimple extends TestCase {
Line 3      public TestSimple(String name) {
Line 4          super(name);
Line 5      }
Line 6      public void testAdd() {
Line 7          assertEquals(2, 1+1);
Line 8      }
Line 9  }
```

- The code above described a simple piece of test code that illustrates the minimum framework you need to get started.
 - First, the import statement on line 1 brings in the necessary JUnit classes.
 - Line 2 is the class definition, where base class TestCase provides most of the unit-testing functionalities
 - Line 3 calls a super to pass a name string to the base class
 - Line 6 shows a test method called testAdd,
 - all methods that starts with test will be run by JUnit automatically



JUnit Test Composition

```
import junit.framework.*;
public class TestClassOne extends TestCase {
    public TestClassOne(String method) {
        super(method);
    }
    public void testAddition() {
        assertEquals(4, 2+2);
    }
    public void testSubtraction() {
        assertEquals(0, 2-2);
    }
}
```

- The code above described a simple piece of test code that illustrates the minimum framework you need to get started.
 - First, the import statement on line 1 brings in the necessary JUnit classes.
 - Line 2 is the class definition, where base class TestCase provides most of the unit-testing functionalities
 - Line 3 calls a super to pass a name string to the base class

What To Test?



**UNIVERSITY OF
MPUMALANGA**



Introduction

- The complexity of testing is associated with the following
 - Looking at a method or a class and try to come up with all the ways it might fail
 - Anticipate all the bugs that might be lurking in there.
- End uses will always find bugs
 - This is embarrassing and damaging to the career of the developer.
- Experience is vital in anticipating things that are likely to break
- How do you identify possible failure modes without experience?
- This section is about finding some guidelines, some reminders of areas that might be important to test.



The Right-BICEP

- There are six specific fundamental areas to test that: **RIGHT-BICEP**
 - **Right**: Are the results right?
 - **B**: Are all the boundary conditions CORRECT?
 - **I**: Can you check inverse relationships?
 - **C**: Can you cross-check results using other means?
 - **E**: Can you force error conditions to happen?
 - **P**: Are performance characteristics within bounds?



1. Right: Are the results Right

- The area to test is simply to see if the expected results are right
 - This means validating the results.

```
public void testLargestSimple() {  
    assertEquals(9, Largest.largest(new int[] {7,8,9}));  
}
```

- This is a simple data validation that must be specified in the requirements. If not specified, you will need to ask someone.
- The question is:
 - If the code ran correctly, how would I know?
- There are time when embracing agility, you might want to invent some as a stake in the ground. It might be a wrong invention, but at least you will know wat to say.

1. Right: Are the results Right

Using Data Files



- In some cases you need to test with a large amount of test data
 - Meaning you need to put the test values in a separate file



2. B: Boundary conditions CORRECT?

- It is valuable to identify the boundary conditions in unit test
 - Because most bugs come from not dealing with boundary conditions
- Example of conditions:
 - Totally bogus or inconsistent input values: "!*W:Xn&Gi/w>g/h#WQ@".
 - Badly formatted data: "email - fred@foobar."
 - Empty or missing values: 0, 0:0, "", or null
 - Values far in excess of reasonable expectations: a person's age of 10,000 years.
 - Duplicates in lists that shouldn't have duplicates: date of birth 1980 and age 1980
 - Ordered lists that aren't, and vice-versa: Try handing a pre-sorted list to a sort algorithm
 - Things that arrive/happen out of order: trying to print a document before logging in



2. B: Boundary conditions CORRECT?

- Use the acronym CORRECT to understand boundary
 - Conformance: Does the value conform to an expected format?
 - Ordering: Is the set of values ordered or unordered as appropriate?
 - Range: Is the value within reasonable minimum and maximum values?
 - Reference: Does the code reference anything external that isn't under direct control of the code itself?
 - Existence: Does the value exist (e.g., is non-null, nonzero, present in a set, etc.)?
 - Cardinality: Are there exactly enough values?
 - Time (absolute and relative): Is everything happening in order? In time?



3. I: Inverse relationships check?

- Some methods can be checked by applying their logical inverse.
- Eg. Checking a method that calculates a square root by squaring the result, and testing that it is tolerably close to the original number:

```
public void testSquareRootUsingInverse() {  
    double x = mySquareRoot(4.0);  
    assertEquals(4.0, x * x, 0.0001);  
}
```



4. C: Cross-check results using other means?

- Try testing through cross-check using other means
- Usually there is more than one way to calculate some quantity;
 - we might pick one algorithm over the others because it performs better,
 - or has other desirable characteristics. That's

```
public void testSquareRootUsingStd() {  
    double number = 3880900.0;  
    double root1 = mySquareRoot(number);  
    double root2 = Math.sqrt(number);  
    assertEquals(root2, root1, 0.0001);  
}
```

- Alternatively, use different pieces of data from the class itself to make sure they all add up
 - Eg. Library system of books in and books out

5. E: Can you force error conditions to happen?



- In the real world, errors happen.
 - Disks fill up, network lines drop, e-mail goes into a black hole, electricity load shedding is not adhered to and programs crash.
- You should be able to test that your code handles all of these realworld problems by forcing errors to occur.
- Invalid parameters and the like are easy
- To simulate specific network errors without unplugging any cables takes some special techniques.
- Solution will be to use Mock Objects, details are beyond the

5. E: Can you force error conditions to happen?

Examples of environmental forced error source



1. Here are a few environmental things we've thought of.
2. Running out of memory
3. Running out of disk space
4. Issues with wall-clock time
5. Network availability and errors
6. System load
7. Limited colour palette
8. Very high or very low video resolution

6. P: Are performance characteristics within bounds?



- One area that might prove beneficial to examine is performance characteristics
 - Don't confuse this with performance itself,
 - Think about problems become more complex, and so on
- We might release one version of the system that works okay,
 - but somehow by the next release it has become dead-dog slow.
 - We don't know why, or what change was made, or when, or who did it
 - And the end users are screaming bloody murder.

6. P: Are performance characteristics within bounds?



- To avoid that awkward scenario, you might consider some rough tests just to make sure that the performance curve remains stable.
 - For instance, suppose we've written a filter that identifies web sites that we wish to block
 - using our new product to view naughty pictures might get us in all sorts of legal trouble, after all